

Web Scraping Cheatsheet (Python)

A reference of every pattern used across the 8-task web scraping project, with the reasoning behind each one — not just *what* the code does, but *why* it's written that way.

1. Fetching pages with `requests`

Basic GET request

```
import requests

response = requests.get(url, headers=headers, timeout=timeout)
```

- `headers` : a dict, most commonly `{"User-Agent": "..."} . Many sites block or rate-limit requests that don't look like they're coming from a real browser (the default requests User-Agent literally says python-requests/x.x , a dead giveaway).`
- `timeout` : seconds to wait before giving up. Without it, a hung server can freeze your script indefinitely.

Always check for errors

```
response.raise_for_status()
```

`requests` does **not** raise an error just because a page returned 404 or 500 — as far as the HTTP layer is concerned, that's still a "successful" exchange. `raise_for_status()` is what turns a bad status code (≥ 400) into an actual Python exception, so failures don't silently propagate as if they were valid data.

Getting the content

```
response.text    # decoded string (HTML, JSON-as-text, etc.)
response.content # raw bytes (for images, files)
```

A realistic browser User-Agent (avoids bot-blocking)

```
HEADERS = {
    "User-Agent": (
        "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
        "AppleWebKit/537.36 (KHTML, like Gecko) "
        "Chrome/115.0.0.0 Safari/537.36"
    )
}
```

Why it matters: a script hammering a site with the default `python-requests` User-Agent, especially many requests in quick succession with no delay, is a classic trigger for rate-limiting or outright connection refusal partway through a run. Spoofing a browser User-Agent is the standard first fix.

Being polite between requests

```
import time
time.sleep(1) # between page fetches in a loop
```

Reduces load on the target server and lowers the chance of getting rate-limited or blocked mid-scrape.

Persisting cookies across multiple requests (logins)

```
session = requests.Session()
session.get(...) # cookies from this response are remembered
session.post(...) # automatically sent with the saved cookies
```

Plain `requests.get()` / `requests.post()` calls are each a fresh, cookie-less connection — the server has no memory of you between them. A `Session` object behaves like a persistent browser tab: once the server sets a login cookie, every subsequent call through that same session object carries it automatically.

2. Parsing HTML with BeautifulSoup

Setup

```
from bs4 import BeautifulSoup
soup = BeautifulSoup(html, "html.parser")
```

"html.parser" is Python's built-in parser — no extra dependency (like `lxml`) required.

Finding elements

```
soup.find("div", class_="quote")          # FIRST match only, or None
soup.find_all("div", class_="quote")      # ALL matches, as a list
```

Note the trailing underscore in `class_` — plain `class` is a reserved Python keyword, so BeautifulSoup's API had to avoid the collision.

Searching within an already-found element

```
quote_div.find("span", class_="text")
```

Calling `.find()` on a tag object (instead of on `soup` itself) restricts the search to *inside* that tag, not the whole document. Essential when looping through repeated blocks (e.g. one `<div class="quote">` per quote) so you don't accidentally grab data from a different block.

Extracting text vs. attributes

```
tag.get_text()          # visible text content, tags stripped
tag["href"]              # a specific HTML attribute, dict-style access
tag.get_attribute(...)   # NOTE: this is Selenium syntax, not BeautifulSoup!
```

Common mix-up: BeautifulSoup tags use `tag["attr"]` or `tag.get("attr")`; Selenium elements use `element.get_attribute("attr")`. They look similar but are different APIs.

Filtering by attribute instead of class

```
soup.find_all("script", type="application/ld+json")
soup.find("input", attrs={"name": "csrf_token"})
```

Use `attrs={...}` (or a direct keyword like `type=`) when the thing identifying an element is an attribute other than `class`.

List comprehensions for extracting many values at once

```
tags = [tag.get_text() for tag in tag_links]
```

Shorthand for building a new list by transforming each item in another list — here, turning a list of `<a>` tag objects into a list of plain strings.

3. Pagination (HTML "Next" link)

```
from urllib import parse

next_li = soup.find("li", class_="next")
if next_li is None:
    current_url = None          # no more pages — stop the loop
else:
    next_href = next_li.find("a")["href"]    # e.g. "/page/2/" (relative!)
    current_url = parse.urljoin(current_url, next_href)
```

Why `urljoin`: the `href` in pagination links is almost always a *relative* path, not a full URL. `urljoin(base, relative)` correctly combines them:

```
urljoin("https://example.com/", "/page/2/")
# -> "https://example.com/page/2/"
```

Efficiency lesson learned: fetch each page's HTML **once** and reuse that same `soup` object both for extracting data *and* for checking the next-page link. Fetching the same URL twice (once via a data-scraping helper, once again just to check pagination) doubles your total request count — which can trip rate limits or exhaust request budgets on strict test/grading servers.

4. Pagination via a JSON API

```
import json

page = 1
while True:
    url = f"{base_url}/api/quotes?page={page}"
    data = json.loads(fetch_html(url))
    # ...process data["quotes"]...
    if not data["has_next"]:
        break
    page += 1
```

When a site's API tells you directly whether more pages exist (a flag like `has_next`), that's simpler and more reliable than parsing HTML for a "Next" button — no BeautifulSoup needed at all for this part.

`while True:` + `break` is the standard pattern when you don't know the number of iterations ahead of time and the stop condition can only be checked *after* processing something (here, after reading the current page's data).

5. JSON-LD structured data extraction

```
scripts = soup.find_all("script", type="application/ld+json")
for script in scripts:
    try:
        payload = json.loads(script.string)
    except (json.JSONDecodeError, TypeError):
        continue # skip malformed blocks instead of crashing

    entries = payload if isinstance(payload, list) else [payload]
    for entry in entries:
        if entry.get("@type") != "Quote":
            continue
        # ...extract fields...
```

- JSON-LD is structured metadata many sites embed for search engines.
- A single `<script>` block might hold ONE object or a LIST of them — normalize both cases into a list so the rest of the code can loop uniformly.
- `.get("key")` instead of `entry["key"]` avoids a `KeyError` crash if a field is missing; `.get("author", {}).get("name")` chains this so a missing nested object doesn't cause an `AttributeError` either.

Normalizing inconsistent field formats

```
def normalize_tags(keywords):
    if keywords is None:
        return []
    if isinstance(keywords, list):
        return keywords
    return [tag.strip() for tag in keywords.split(",")]
```

Real-world data is often inconsistent (sometimes a list, sometimes a comma-separated string) — write a small helper to normalize it into one predictable shape rather than special-casing it everywhere it's used.

6. Login with CSRF token (session-based auth)

```

session = requests.Session()

login_page = session.get(login_url)
soup = BeautifulSoup(login_page.text, "html.parser")
csrf_token = soup.find("input", attrs={"name": "csrf_token"})["value"]

payload = {"username": user, "password": pwd, "csrf_token": csrf_token}
session.post(login_url, data=payload)

protected_page = session.get(protected_url) # cookie carried automatically

```

A CSRF token is a random, one-time value the server hands out in the login form and only accepts back if it matches — proof the POST really came from that form. You must GET the form first to read the token before you can POST successfully.

7. Selenium: browser automation basics

Setting up headless Chrome

```

from selenium import webdriver

options = webdriver.ChromeOptions()
options.add_argument("--headless")           # no visible browser window
options.add_argument("--window-size=1920,1080") # fixed viewport
options.add_argument("--no-sandbox")          # needed running as root in containers
options.add_argument("--disable-dev-shm-usage") # prevents renderer crashes

driver = webdriver.Chrome(options=options)

```

Why each flag, in practice (lessons learned the hard way): - `--headless` (legacy) vs. `--headless=new`: the newer mode has known intermittent stability bugs (`session not created`, `no such window`, `loader has changed while resolving nodes`) on some Chrome/driver version pairs, especially in constrained sandboxes. **Prefer the legacy `--headless` flag** unless you have a specific reason to use the new one. - `--disable-dev-shm-usage`: containers often allocate a tiny `/dev/shm` (shared memory), which Chrome's renderer relies on heavily. When it runs out, the renderer can crash silently mid-scrape. This flag makes Chrome fall back to disk instead — **almost always worth including by default** in any containerized/CI Selenium setup. - `--window-size=1920,1080`: some pages render differently (or hide elements) at small/default sizes — pinning a common desktop resolution keeps element positions and visibility consistent.

Always clean up the browser process

```

driver = webdriver.Chrome(options=options)
try:
    driver.get(url)
    # ...scraping logic...
finally:
    driver.quit()

```

Put `driver.get(...)` and everything else **inside** the `try` block, not just the cleanup. If `driver.get()` itself throws (bad URL, page timeout, etc.) while outside the `try`, `driver.quit()` never runs — leaving a zombie Chrome process using memory in the background.

8. Selenium: locating and reading elements

Finding one vs. many

```

driver.find_element("class name", "thumbnail") # FIRST match, raises if none found
driver.find_elements("class name", "thumbnail") # ALL matches, as a list (empty if none)

```

Locator strategies (as plain strings)

String	Selects by
<code>"class name"</code>	a single CSS class

"css_selector"	any CSS selector (most flexible)
"id"	element <code>id</code> attribute
"tag_name"	HTML tag name
"xpath"	XPath expression

Important gotcha we hit: `selenium.webdriver.common.by.By` is just a class of string constants (`By.CLASS_NAME == "class name"` , `By.CSS_SELECTOR == "css_selector"` , etc.). If an environment restricts imports to only `time` and `webdriver` , you can — and should — skip the `By` import entirely and pass these raw strings directly. Functionally identical, zero extra import.

Reading data out of an element

```
element.text                # visible rendered text
element.get_attribute("title")  # any HTML attribute, e.g. title, href, data-*
```

Why prefer an attribute over visible text sometimes: visible text can get truncated with `"..."` in the UI for long values, while the underlying attribute (e.g. `title="Full Product Name Here"`) usually holds the complete, untruncated value.

Robust CSS selectors: exact class-token matching

```
driver.find_elements("css_selector", ".ratings [class~='ws-icon-star']")
```

`[class~='token']` matches any element that has that EXACT class token among possibly several classes, regardless of tag name or class order — more robust than chaining `.class1.class2` (which requires an exact class *list* match) when you're not 100% sure which tag wraps an icon or what other classes might be present. This also correctly excludes a similarly-named-but-different class (like an outline/empty-star variant) since `~=` requires a full token match, not a substring.

Counting rather than reading a single value

```
star_elements = driver.find_elements("css_selector", "...")
rating = len(star_elements)
```

Some data isn't stored as a number anywhere — it's represented visually as N repeated icon elements. In that case, the "value" is simply how many matching elements exist, not something read off any single element.

9. Selenium: scrolling / infinite-scroll pages

```
driver.execute_script("return document.body.scrollHeight")
driver.execute_script("window.scrollTo(0, document.body.scrollHeight);")
```

`execute_script()` runs raw JavaScript inside the loaded page, as if typed into the browser console. `scrollHeight` reports the page's total height in pixels — this grows as more content loads in.

The scroll-and-wait loop

```
last_height = driver.execute_script("return document.body.scrollHeight")

while True:
    driver.execute_script("window.scrollTo(0, document.body.scrollHeight);")
    time.sleep(scroll_pause)
    new_height = driver.execute_script("return document.body.scrollHeight")

    if new_height == last_height:
        break # no growth after scrolling = we've hit the true bottom
    last_height = new_height
```

Scrolling to the bottom triggers the page's JS to fetch and append more content. Comparing height before/after each scroll+wait cycle is how you detect "there's nothing left to load" without knowing the page's total size in advance.

Faster alternative: adaptive polling (avoids wasted fixed waits)

```
poll_interval = 0.2
elapsed = 0.0
new_height = last_height
while elapsed < scroll_pause:
    time.sleep(poll_interval)
    elapsed += poll_interval
    new_height = driver.execute_script("return document.body.scrollHeight")
    if new_height != last_height:
        break # content already loaded — stop waiting early
```

Why this matters: a flat `time.sleep(scroll_pause)` after every single scroll wastes time whenever content actually loads faster than the full wait window. Under a strict time budget (e.g. a 30-second checker timeout, many scroll batches needed for 100+ items), this adds up fast. Polling in short intervals and breaking out as soon as new content appears turns `scroll_pause` into a *maximum* wait instead of a *fixed* one — often cutting total runtime by more than half.

Deduplicating results

```
seen = set()
key = (title, price)
if key in seen:
    continue
seen.add(key)
```

On an infinite-scroll page, the same product can sometimes get scraped twice if it appears in more than one loaded batch. A tuple like `("Aspire E1-510", "$306.99")` is hashable, so it can go directly into a `set` for fast "have I seen this before?" checks — much faster than scanning a list of dicts with `in`.

10. Debugging lessons worth remembering

1. **Fetch each page once, not twice.** If you need both the data *and* pagination info from the same page, get one `soup` /response and reuse it — don't call your fetch function twice on the same URL.
2. **Spoof a browser User-Agent** proactively for any real-world site, especially before making many requests in a row.
3. **Comments matter for text-scanning import checkers.** If a grading tool scans raw file text for disallowed import patterns (rather than parsing real Python imports), even a comment mentioning `"importing selenium's By class"` can trigger a false positive. Avoid naming disallowed modules/classes in comments too, not just code.
4. **Function names must match exactly what the test script imports** — a task's written description isn't always 100% in sync with the actual checker file. When in doubt, check the literal `__import__(...)` line in the real checker/test script.
5. `--headless=new` **can be less stable than legacy** `--headless` on some Chrome/driver combinations — worth trying the legacy flag first if you see odd, non-reproducible Selenium session errors.
6. `--disable-dev-shm-usage` **is cheap insurance** against silent renderer crashes in any containerized/CI Selenium environment.
7. **Prefer tag-agnostic, exact-token CSS selectors** (`[class~='...']`) over rigid multi-class chains when you're not 100% certain of a page's exact markup — more resilient to small structural differences.
8. **Under a time budget, poll instead of sleeping a fixed amount** — turn wait parameters into *maximums*, not *fixed* delays, wherever the actual completion time can vary.